

FIAP · Pós-Tech em Arquitetura de Software · Tech Challenge

BlogFIAP

FASE 4 — MOBILE

Interface gráfica nativa em React Native + Expo para gestão pedagógica

Daniel Flávio · Washington Barsotti

Julho de 2026 — Apresentação e Defesa do Projeto

Agenda & Roteiro

01

O Desafio & Solução

Contexto pedagógico e evolução para o ecossistema Mobile RN

02

Arquitetura & Stack

React Native 0.81, Expo 54, Node.js e PostgreSQL em Docker

03

Requisitos Atendidos

Mapeamento integral das 14 telas, busca, paginação e 4 CRUDs

04

Autenticação & Permissões

Acesso protegido (useAuthGuard) e papéis de Professor vs. Aluno

05

Qualidade & Destaques

Auto-migration, UI com paleta pastel e prontidão de entrega

O Desafio & A Solução

O Problema (Contexto)

Docentes enfrentam dificuldades crônicas na centralização e distribuição ágil de conteúdos pedagógicos.

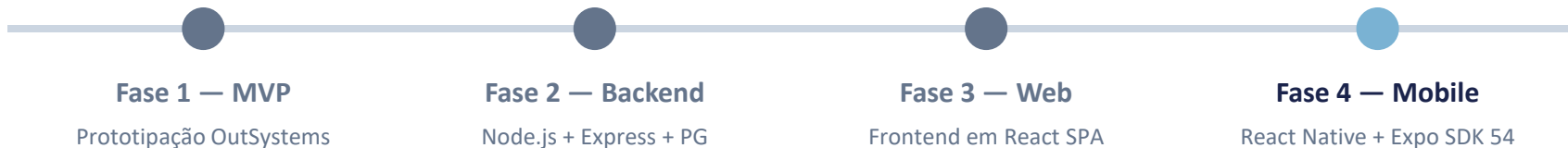
A ausência de plataformas móveis modernas limita o engajamento de alunos e cria barreiras no acesso ao material didático no dia a dia.

A Solução Mobile (Fase 4)

Aplicativo React Native + Expo conectado à API REST Node.js, com interface fluida e paleta pastel acolhedora.

Oferece leitura e busca instantânea de posts, além de painel administrativo completo e protegido com JWT para docentes.

Evolução do Ecosistema BlogFIAP



Arquitetura & Stack Tecnológica

FRONTEND MOBILE

React Native 0.81 · Expo SDK 54 · Axios
React Navigation 7 (Native Stack) · Context API

▼ HTTP / REST + Authorization Bearer Token

BACKEND API REST

Node.js 20 · Express 5 · CORS
Autenticação JWT · Hash bcryptjs (salt 10)

▼ Conexão Pelo Pool (pg) + ensureTables()

BANCO DE DADOS

PostgreSQL 15 em Container Docker
node-postgres (pg pool max 5) · Auto-Migration

Destaques da Arquitetura

- Cross-Platform Unificado: Executa com perfeição no Android, iOS e Web via React Native Web
- Detecção Automática de Host: Resolve 10.0.2.2 no emulador e localhost em navegadores
- Estilização Yoga Layout: Flexbox nativo, bordas suaves e paleta pastel coesa em 14 telas
- Ambiente 100% Reproduzível: Orquestrado via Docker Compose com healthcheck
- Gestão de Estado Leve: Context API nativa (AuthContext) sem overhead de bibliotecas externas

Cobertura 100% dos Requisitos Funcionais

Req 1 & 2

Feed & Leitura Completa

Lista paginada com busca por palavra-chave + tela de post com autor e data

Req 3, 4, 9

CRUD de Postagens

Formulários para criar, editar, listar e excluir postagens no painel admin

Req 5, 6, 7

Gestão de Professores

Tabela paginada com cadastro, edição, exclusão e hash bcrypt automático

Req 8

Gestão de Estudantes

Replicação arquitetural completa das 4 telas de CRUD para alunos

Req 10

Autenticação & Autorização

Login JWT com proteção de rotas (useAuthGuard) em todas as telas admin

Req Técnico

Context API & Interceptors

Estado global de auth e injeção automática de token via Axios Interceptors

Autenticação & Permissões de Acesso

Acesso Público (Estudantes / Visitantes)

- Home (Feed Paginado): Visualização imediata de todos os posts sem necessidade de login
- Busca por Palavra-Chave: Filtro dinâmico na barra superior consultando `/posts/search?q=`
- Leitura Completa (PostDetail): Acesso ao conteúdo integral, autor e carimbo de data/hora
- Segurança de Mutabilidade: Alunos e visitantes não visualizam nem acessam botões de criar, editar ou excluir



Painel Administrativo (Docentes)

- Proteção `useAuthGuard()`: Redireciona automaticamente para Home se não houver token ativo
- Gestão Total de Posts: Criação, edição com dados pré-carregados e remoção via API
- Administração de Docentes: Cadastro de novos colegas com criptografia `bcrypt` automática
- Administração de Alunos: Tabela paginada com ações de editar e excluir estudantes

Endpoints da API REST Integrados

Método	Endpoint	Ação e Comportamento no Mobile	Auth JWT
POST	/auth/login	Autentica e-mail + senha com bcrypt e retorna Token JWT 24h	Público
GET	/posts	Lista paginada de postagens exibidas no feed principal (Home)	Público
GET	/posts/search?q=	Busca filtrando posts em tempo real por palavra-chave	Público
POST	/posts	Cria nova postagem associada ao autor e grava no banco	 Sim
PUT	/posts/:id	Atualiza título, conteúdo e autor do post selecionado	 Sim
DELETE	/posts/:id	Remove permanentemente a postagem e atualiza a lista	 Sim
GET	/professores /alunos	Lista docentes e alunos para tabelas gerenciais paginadas	 Sim
POST	/professores /alunos	Cadastra docentes (hash de senha) ou novos estudantes	 Sim
PUT / DELETE	/professores/:id /alunos/:id	Edita dados cadastrais ou exclui o registro selecionado	 Sim

Destques Técnicos de Implementação

Auto-Migration no Startup (ensureTables)

Para eliminar erros de deploy, o backend executa `ensureTables()` a cada inicialização: cria tabelas automaticamente (IF NOT EXISTS) e adiciona colunas faltantes de forma idempotente sem downtime.

Seed Seguro via bcryptjs (Salt 10)

O sistema garante a existência do usuário administrador padrão, realizando o hash criptográfico da senha na subida do container. O banco nunca armazena senhas em texto plano.

Navegação Fluida e Protegida (RN 7)

Arquitetado sobre React Navigation Native Stack, garantindo transições nativas de 60 FPS. O hook `useAuthGuard()` protege rotas administrativas e `reset()` limpa a pilha no logout.

Design System com Paleta Pastel

Estilizado 100% via `StyleSheet.create()` com Yoga Layout (Flexbox). Componentes visuais suaves com cantos arredondados, sombras consistentes e alta legibilidade em todas as telas.

Qualidade, Testes & Entregáveis

14

Telas Integradas

4

CRUDs Completos

10

Requisitos Atendidos

100%

Conclusão do Projeto

Checklist Oficial de Entregáveis da Fase 4

#	Entregável Exigido	Formato & Detalhes	Status
1	Código-Fonte Completo (Mobile + API + Docker)	Repositório GitHub estruturado com histórico limpo	✓ Concluído
2	Apresentação Gravada em Vídeo (até 15 min)	Demonstração prática ao vivo + explicação técnica (este slide)	✓ Pronto p/ Gravar
3	Documentação Técnica & Relato de Experiência	README.md detalhado + Documento DOCX gerado e validado	✓ Concluído

Desafios Superados & Lições Aprendidas

Desafios Enfrentados

- Conectividade Mobile ↔ API: Emuladores Android requerem 10.0.2.2 enquanto dispositivos físicos necessitam de IP LAN na mesma rede.
- Evolução de Schema sem Perda de Dados: Criar colunas adicionais no PostgreSQL em produção sem quebrar contêineres em execução.
- Sincronização de Estado Auth: Garantir que a pilha de navegação reaja instantaneamente nas transições de Login/Logout.

Lições Aprendidas

- Docker Compose Orquestrado: Padronizou todo o ambiente, reduzindo o tempo de setup local da equipe a poucos segundos.
- Consistência Cross-Platform via Expo: Expo SDK 54 provou ser excepcional, permitindo que o mesmo código execute no Web e Mobile.
- Context API + Hooks: Estrutura leve e coesa que atendeu perfeitamente todos os requisitos de autenticação sem complexidade extra.

Obrigado pela atenção! — BlogFIAP Fase 4

github.com/danflavio/postech-fiap-etapa-4 • Daniel Flávio & Washington Barsotti